Kafka Cost Control

Table of Contents

1. User Manual . . . 3
   1.1. Introduction . . . 4
      1.1.1. Graphql . . . 4
      1.1.2. Sign-in . . . 4
   1.2. Config samples . . . 5
   1.3. Configuring your cost control application . . . 6
      1.3.1. Configuring aggregation types . . . 6
      1.3.2. Configuring transformations . . . 6
      1.3.3. Configuring clusters . . . 7
   1.4. Pricing rules . . . 9
      1.4.1. Pricing rules or the invoice split . . . 9
      1.4.2. Listing pricing rules . . . 9
      1.4.3. Setting a pricing rule . . . 10
      1.4.4. Price changes . . . 10
      1.4.5. Removing a pricing rule . . . 11
   1.5. Monthly bills . . . 12
      1.5.1. How a bill is shared . . . 12
      1.5.2. How costs are computed . . . 12
      1.5.3. Applying a bill’s rates to the pricing rules . . . 13
      1.5.4. Storage . . . 13
   1.6. Context data . . . 15
      1.6.1. Listing existing context data . . . 15
      1.6.2. Setting context data . . . 16
      1.6.3. Removing context data . . . 16
   1.7. Reprocess . . . 18
      1.7.1. Using the UI . . . 18
   1.8. Deleting a metric . . . 19
      1.8.1. Using the UI . . . 19
      1.8.2. Using GraphQL . . . 19
2. Installation . . . 20
   2.1. Prerequisites . . . 21
   2.2. Topics and AVRO schemas . . . 22
      2.2.1. Reference AVRO schemas . . . 22
      2.2.2. Topics . . . 22
   2.3. Kubernetes . . . 24
      2.3.1. Kafka metric scraper . . . 24

2.3.2. Kafka cost control . . . 25
2.3.3. Helm Chart (Strimzi only) . . . 26
2.4. Metric storage . . . 27
2.4.1. DuckDB Integration . . . 27
2.5. Troubleshooting . . . 29
2.5.1. Kafka cost control is never ready . . . 29
3. Architecture . . . 30
3.1. Introduction and Goals . . . 31
3.1.1. Requirements Overview . . . 31
3.1.2. Quality Goals . . . 32
3.1.3. Stakeholders . . . 33
3.2. Architecture Constraints . . . 34
3.3. System Scope and Context . . . 35
3.4. Solution Strategy . . . 36
3.4.1. Used Technologies . . . 36
3.4.2. Time based aggregations & scraping intervals . . . 36
3.5. Building Block View . . . 37
3.5.1. Whitebox Overall System . . . 37
3.5.2. MetricsScraper . . . 37
3.5.3. PricingRules . . . 38
3.5.4. ContextProvider . . . 38
3.6. Runtime View . . . 41
3.6.1. Metrics Ingestion from Confluent Cloud . . . 41
3.6.2. Metrics using Kafka Admin API . . . 41
3.6.3. Other sources of metrics . . . 41
3.6.4. Metrics Enrichment . . . 42
3.6.5. Metrics Grouping . . . 43
3.7. Deployment View . . . 44
3.8. Risks and Technical Debts . . . 45
3.9. Glossary . . . 46

1. User Manual

1.1. Introduction

This user manual will help you understand kafka Cost Control and how to use it propertly. This document assumes that you already have a running application. If not please see the Installation section.

At this point you should have access to the Kafka Cost Control UI.

1.1.1. Graphql

Kafka cost control provides a graphql endpoint at: *<your-host>/graphql-ui*

In addition, there is a ready to use GraphQL UI. You can access it by going to the following URL: *<your-host>/graphql-ui*

1.1.2. Sign-in

Kafka Cost Control is either open to everyone or closed to everyone who hasn't signed in; there is no part in between. `CC_AUTH_MODE` decides how:

- `basic` (default): sign in as `admin` with the password in `CC_ADMIN_PASSWORD` (default `admin`, so set it).
- `oidc`: sign in through an OpenID Connect provider, e.g. Google, Microsoft Entra ID or Keycloak. The `admin` user still works with basic auth, for scripts such as the pricing-rule loader.
- `none`: no sign-in, for example when a proxy in front of Kafka Cost Control already authenticates every request. Everyone who reaches it can also change rules and start a reprocess.

The health (`/q/health`) and metrics (`/q/metrics`) endpoints stay open in every mode.

OpenID Connect

Register Kafka Cost Control as a web application (confidential client) at your provider, with the redirect URI `https://<your-host>/auth/callback`, and set:

```
CC_AUTH_MODE=oidc
QUARKUS_OIDC_PROVIDER=google            # or
QUARKUS_OIDC_AUTH_SERVER_URL=https://<issuer>
QUARKUS_OIDC_CLIENT_ID=<client id>
QUARKUS_OIDC_CREDENTIALS_SECRET=<client secret>
CC_AUTH_ALLOWED_DOMAINS=example.com     # who may sign in; comma-separated
```

`CC_AUTH_ALLOWED_DOMAINS` (and `CC_AUTH_ALLOWED_EMAILS` for single addresses) limits who gets in by the e-mail the provider confirms. Leave both empty only if the provider holds nobody but your own users: anyone with a Google or Microsoft account can sign in to a Google or Microsoft client.

The sign-in runs in the aggregator, so `/auth` must reach it on the same host as the UI, like `/graphql` (the Helm chart and the Kubernetes manifests route it). Behind a reverse proxy, the proxy must send `X-Forwarded-Proto` and `X-Forwarded-Host`, so the redirect URI uses the external address.

1.2. Config samples

If you want to quickly get started, you can create pricing rules and context data using the config sample folder. All you need is node JS 20+.

Be sure to edit the files `pricing-rules.json` and `context-data.json` to match your environment.

To persist the configuration you can use the following command:

```
node index.js --url=https://<your-host>/graphql --user=admin --password=your-password
```

If you have issues you can try to add the `--verbose` options. This will display all the requests.

1.3. Configuring your cost control application

Your cost control instance can be configured either by setting environment variables, or (if you are using one of the provided container images) by mounting an `application.properties` file at `/deployments/config/application.properties`:

```
# inside your application.properties
quarkus.profile=ccloud
```

1.3.1. Configuring aggregation types

In cost control, each metric is associated with an aggregation type. The aggregation type determines how multiple measurements within a time window are combined to produce a single value. Possible aggregation types are `SUM` and `MAX`. If no aggregation type is specified for a metric, then `SUM` will be used. To specify an aggregation type for a metric, add a line to your `application.properties` file like this:

```
# inside your application.properties
cc.metrics.aggregations.<metric-name>=max
```

For example, to aggregate retained byte measurements using the `MAX` aggregation type, add the following line to your `application.properties` file:

```
cc.metrics.aggregations.confluent_kafka_server_retained_bytes=max
```

Note that these settings may only be set in an `application.properties` file, and not via environment variables. This is due to the fact that Quarkus cannot unambiguously map environment variable names to property names if said property names contain user-defined parts (like `confluent_kafka_server_retained_bytes` in the example above). For more information, see the relevant section of the Quarkus documentation.

1.3.2. Configuring transformations

Currently, there is one kind of transformation that can be added to the metrics processing pipeline: `splitTopicMetricAmongPrincipals`. This transformation is configured by supplying a map of metric names to context keys in application.properties. For example:

```
cc.metrics.transformations.splitMetricAmongPrincipals={bytesin:'writers',bytesout:'rea
ders'}
```

This will expect `bytesin` metrics to have a `writers` key in their context, which should be a comma-separated list of principals (e.g. applications, teams, ...) that can write to this topic. When a `bytesin` metric for any topic is encountered, the metric will be replaced with `n` metrics (where `n` is the length of the `writers` list in that metric's context). The name of the topic will be moved into that metric's

context under the `topic` key and the name of a principal from the `writers` list will move into the metric's `name` field. The value of each generated metric will be the value of the original metric divided by `n`.

In effect this takes a single metric that says "90 kb have been written to topic ABC by writers X,Y,Z" and transforms it into 3 metrics:

- Principal X wrote 30 kb to topic ABC
- Principal Y wrote 30 kb to topic ABC
- Principal Z wrote 30 kb to topic ABC

The advantage over the original metric is that each such transformed metric will be put into its own database row, which will make it easier to aggregate by principal (e.g. answer questions like "how much did principal X produce in the past month"). This type of transformation is especially relevant when doing cost control based on vanilla Kafka broker metrics (as is the case in Strimzi deployments, for example), because here only topic-level metrics are generated natively. This transformation allows to turn these topic-focused metrics into principal-focused ones.

Some metrics that are configured this way may not have a matching context key in the context. To handle such cases, cost control offers three different strategies that can be enabled by setting the `cc.metrics.transformations.config.splitMetricAmongPrincipals.missingKeyHandling` property in `application.properties`:

- `ASSIGN_TO_FALLBACK`: The metric will be handled as if the context key existed and contained a single principal name. This fallback principal name is set to "unknown" per default but can be changed by setting the `metrics.transformations.config.splitMetricAmongPrincipals.fallbackPrincipal` property in `application.properties`.
- `DROP`: The metric will not be forwarded downstream and will be dropped.
- `PASS_THROUGH`: The metric will be passed downstream without any changes. This is the default behavior.

Note that `cc.metrics.transformations.config.splitMetricAmongPrincipals.missingKeyHandling` is expected to be a map of metric names to one of the above strategies. For example:

```
cc.metrics.transformations.config.splitMetricAmongPrincipals.missingKeyHandling={bytes
in:'ASSIGN_TO_FALLBACK',bytesout:'DROP'}
```

If no strategy is specified for a metric, then `ASSIGN_TO_FALLBACK` will be used.

1.3.3. Configuring clusters

One cost control instance can serve several Kafka clusters. Each metric keeps the cluster it came from as the context value `cluster`, so the same topic or principal on two clusters stays two rows, and every view that groups or filters by context can do so by cluster. No context rule is needed: the cluster comes from a tag the raw metrics already carry.

- Confluent Cloud's metrics export and the kafka-scraper tag every metric with `kafka_id`, the cluster's ID (`lkc-⋯`).
- The Strimzi Helm chart's Telegraf adds `kafka_id` from `strimzi.clusterName`, and the chart sets the same name as `cc.cluster.default`. For another collector, add a global tag.

All settings are optional:

Property	Default	Meaning
`cc.cluster.tag`	`kafka_id`	The raw metric's tag that names its cluster.
`cc.cluster.context-key`	`cluster`	The context key the cluster is written to. It replaces a value a context rule set for the same key.
`cc.cluster.default`	(unset)	The cluster of metrics without the tag. Unset, they get no cluster.
`cc.cluster.names`	(unset)	Readable names for cluster IDs, as `id=name` pairs separated by commas. IDs without a name are shown as they are.

For example, as environment variables:

```
CC_CLUSTER_NAMES=lkc-a1b2=confluent-prod,lkc-c3d4=confluent-dev
```

Rows written before an upgrade to a release with this feature have no cluster. A reprocess (see the section on reprocessing) fills it in for the hours the raw topic still holds. It also tidies the hour the upgrade happened in: rows from before and after the restart are two rows there, which double that hour of metrics aggregated as `max`, like retained bytes.

1.4. Pricing rules

Pricing rules are a way to put a price on each metric. The price will be applied on the **hourly** aggregate. Also, it's common for metrics to be in **bytes** and not Megabyte or Gigabyte. Keep that in mind when setting the price. For example, if you want to have a price of *1.0$* per *GB* you will need to set the price to *1.0/1024*^{3} *= 0.000000000931$* per *byte*. You don't have to do this conversion: a rule can carry a price per GB, per GB-hour or per unit, optionally times a multiplier such as 3 replicas, and KCC derives the per-byte factor (see *Setting a pricing rule*). The pricing rules tab shows every rule as a price, e.g. *$0.00012603 per GB-hour × 3 replicas*.

Pricing rules are stored in kafka in a compacted topic. The key should be the metric name.

A metric without a pricing rule gets no cost, so costs from pricing rules leave it out. The pricing rules tab lists the collected metrics that have no rule.

Nothing re-creates rules on its own: restarts and redeployments keep them. The sample loader (`config-sample`) saves every rule in its file when you run it, so running it again overwrites rules with the same metric name, including ones changed or deleted in the UI.

1.4.1. Pricing rules or the invoice split

Kafka Cost Control has two ways to put costs on topics and principals:

- **Pricing rules** are a rate card: every window costs `baseCost + costFactor x value`, per entity. Use them where there is no bill to share (e.g. a self-hosted cluster), or to estimate.
- **Monthly bills** share what the provider really billed among the entities, in proportion to their usage (see *Monthly bills*). Where no bill applies, the pricing rules fill in, marked as estimated.

Free tiers, commitments and discounts are not part of pricing rules: the bill already contains them. For example, Confluent Cloud bills partitions beyond the first 10 per cluster; a rule at $0.0046 per partition-hour charges all of them at list price, while a monthly bill's partition line is already net of the free ones. For the costs you actually pay, enter the bills.

1.4.2. Listing pricing rules

From the UI

Simply go to the pricing rules tab of the UI. You should see the metric name and its cost.

Using Graphql

```
query getAllRules {
  pricingRules {
    creationTime
    metricName
    baseCost
    costFactor
  }
```

```
}
```

1.4.3. Setting a pricing rule

From the UI

Sign in, go to the pricing rules tab and click *Add pricing rule*, or the edit button of an existing rule. A metric without a rule can also be priced straight from the list of metrics without a pricing rule. Saving a rule for a metric that already has one replaces it.

Using Graphql

```
mutation saveRule {
  savePricingRule(
    request: {
      metricName: "confluent_kafka_server_retained_bytes"
      baseCost: 0
      price: 0.00012603
      priceUnit: GB_HOUR
      multiplier: 3
      multiplierLabel: "replicas"
    }
  ) {
    metricName
    costFactor
    price
    priceUnit
    multiplier
  }
}
```

`priceUnit` is what the price is per:

- `GB`: per GB of the metric's value, 1 GB = 1024^3 bytes (e.g. network bytes);
- `GB_HOUR`: per GB held for one hour, for stored-bytes metrics, whose hourly maximum is the GB-hours a provider bills;
- `UNIT`: per unit of the metric's value for one hour (e.g. per partition).

The optional `multiplier` applies on top of the price, e.g. `3` for storage that Confluent Cloud bills per replica. KCC derives the `costFactor` it applies from these: `price × multiplier ÷ bytes per unit`. You can still send a bare `costFactor` (per unit of the raw value) instead of a price.

1.4.4. Price changes

A rule can have a price history. Saving a rule either:

- **corrects** its current price (no `validFrom`): the new price applies everywhere the current one did, e.g. to fix a mistake;

- or sets a **new price from a date** (`validFrom`, the start of a day in UTC such as `2026-11-01T00:00:00Z`): the current price keeps the days before it, the new one applies from it on. Use this when the provider's price changes, usually on the 1st of a month.

Each hour is priced with the price valid at its start. A bill's rates can be saved as new prices from the Bills page: see "Applying a bill's rates to the pricing rules" under Monthly bills. `undoPricingRuleChange(request: {metricName: "…"})` drops the current price and makes the previous one current again.

```
mutation newPrice {
  savePricingRule(request: {metricName: "confluent_kafka_server_request_bytes",
baseCost: 0,
      price: 0.16, priceUnit: GB, validFrom: "2026-11-01T00:00:00Z"}) {
    metricName
    validFrom
    earlierPrices { validFrom validUntil price priceUnit }
  }
}
```

1.4.5. Removing a pricing rule

From the UI

Sign in, go to the pricing rules tab and click the delete button of the rule.

Using Graphql

```
mutation deleteRule {
  deletePricingRule(request: {metricName: "whatever"}) {
    creationTime
    metricName
    baseCost
    costFactor
  }
}
```

1.5. Monthly bills

Enter what your provider billed for a month, and Kafka Cost Control shares it among topics and principals by their usage. Where no bill applies, the costs come from the pricing rules and are marked as estimated - so a running month shows estimates until its bill is in.

A bill has, in dollars:

- one amount per usage-billed line, named after the metric it is shared by: `confluent_kafka_server_request_bytes` (network write), `confluent_kafka_server_response_bytes` (network read), `confluent_kafka_server_retained_bytes` (storage) and `kafka_topic_partition_count` (partitions);
- optionally **other lines**: everything else on the bill (connectors, support, credits as a negative amount), each with a description and where it goes:
  - **to a context**, e.g. `tenant=data-platform` or `application=etl`: all of it, shown under that context;
  - **spread by usage**: shared out like overhead, in proportion to each group's usage-based costs that month;
  - **shared**: assigned to no one, shown as `<shared>`;
- optionally **covers up to**: for a month-to-date bill, where its amounts stop. After that, the month is estimated.

All other lines appear under the cost `other`.

Saving a bill for a month that already has one replaces it. A line you leave empty is estimated from the pricing rules.

1.5.1. How a bill is shared

Each month is taken on its own (UTC). A topic's part of a line is its usage in the month divided by the month's usage, up to where the bill stops. For a date range that covers only part of a month, that is the usage in the range: 10 to 20 September gets the usage of those days divided by September's usage, times September's line. A range across months adds up each month's part of its own bill.

So costs follow how much was really used, not the number of days. Nothing needs to be recalculated when a bill is added or corrected: the costs are computed when you look at them. (Context rules are different: a change there needs a reprocess.)

If Kafka Cost Control has data for only part of a month, the month's bill is shared over what it has. A bill line for usage it hasn't measured at all goes to `other` as `<shared>`.

1.5.2. How costs are computed

Every place that shows a cost - Cost Overview, Explore, the unassigned topics and service accounts - reads the same view, `costs`, in the OLAP database. It gives each stored hour:

- in a month with a bill: the bill line's share by usage, as above; a metric that isn't one of the bill's

lines costs nothing there, since everything else the provider billed is in the bill's other lines;

- otherwise - no bill, after a month-to-date bill stops, or a line the bill leaves empty - the pricing rule's cost for that hour, marked as estimated;
- and the bill's other lines and unmeasured lines, under `other`.

The view is rebuilt when the rules or bills change (at the latest after 10 seconds), so it always reflects them.

The OLAP export (`/olap/export`) reads the view too: each row has `cost`, `rate_cost` (the pricing rule's), `estimated` and `shared`, and the bill's other lines are rows under `other`.

The `aggregated` and `aggregated-table-friendly` topics carry usage only: their `cost` field stays in the schema but is empty. A consumer that wants costs reads them from Kafka Cost Control (the export or the GraphQL API). Versions before 0.11 filled it with the pricing rule's cost when the hour was processed, which a later bill or price change never updated.

1.5.3. Applying a bill's rates to the pricing rules

A list price rarely matches what the provider charges for what Kafka Cost Control measures: free allowances (Confluent's first partitions), storage billed per replica, metrics that count a little low. A bill already holds the real rates. On the **Bills** page, a bill's **Apply the bill's rates to the pricing rules** button shows, for each of its usage lines, the billed amount, the usage measured in the hours the bill covers, and the rate - amount divided by usage - as a price in the rule's unit, next to the current price.

Saving the ticked lines sets the pricing rules to these rates, by default as a **new price from the day the bill ends** (the 1st of the next month for a whole month): the hours after the bill are estimated at what the provider really charged, and earlier hours without a bill keep their prices. **Correct the current prices** applies them every day the current prices do instead. Hours a bill covers keep their billed cost either way. A rule keeps its unit and multiplier; its base cost is set to 0, since the rate alone reproduces the bill. A line without measured usage has no rate.

The rates are also available as a query: `billRates(month: "2026-10") { metricName amount usage costFactor }`, where `costFactor` is the cost per unit of the metric's raw value (per byte, per partition-hour).

1.5.4. Storage

Bills are kept in a compacted Kafka topic (`cc.topics.bills`, default `bills`), keyed by month. Kafka Cost Control creates it when it is missing; where its user may not create topics, create it beforehand with `cleanup.policy=compact` and one partition. Until the topic can be read, bills are unavailable and the rest of the application works as before.

Using GraphQL

```
mutation saveBill {
  saveBill(request: {month: "2026-09", networkWrite: 0.27, networkRead: 0.22, storage:
8.12, partitions: 22.50,
```

```
      otherLines: [{description: "Support", amount: 50, allocation: USAGE},
                   {description: "Connect", amount: 12, allocation: CONTEXT, context:
[{key: "application", value: "etl"}]}]}) {
    month
    updatedBy
  }
}

query costs {
  billedCosts(request: {from: "2026-09-10T00:00:00Z", to: "2026-10-10T00:00:00Z",
contextKeysToGroupBy: ["tenant"]}) {
    metrics { metric shares { name price estimatedPrice } }
    months { month billed billedUntil }
  }
}
```

`billedCosts` returns cents. `estimatedPrice` is the part of `price` that comes from the pricing rules.

1.6. Context data

Context data are a way to attach a context (attributes basically) to a kafka item (topic, principal, ...). Basically define a set of key/values for an item that match a regex. It is possible that one item match multiple regex (and thus multiple context), but in this case you have to be careful to not have conflicting key/values.

A principal rule's regex is matched against the principal's ID and, when the metrics carry one, its display name (Confluent Cloud sends `principal_name` next to `principal_id`). So instead of listing service account IDs, one naming-convention rule such as `^kcc-demo-()\.app\.(.)$` with `tenant: $1` and `application: $2` covers every account that follows the convention, including ones created later. Capture groups come from whichever of the two matched.

You can have as much key/values as you want. They will be used to sum up prices in the dashboard. It is therefor important that you have at least one key/value that defined the cost unit or organization unit. For example: `organzation_unit=department1`.

The cluster a metric came from needs no rule: it is added as the context value `cluster` (see the section on configuring clusters). A rule matches by name only, so it applies on every cluster.

The context data are stored in kafka in a compacted topic. The key is free for the user to choose.

1.6.1. Listing existing context data

From the UI

Go to the tab *Context Data* in the UI. You should see all the context with their validity time, type, regex and context key/values.

If you are signed in, you can use the context tester via the button *test context data* to check what context key/values currently apply to your topic or principal.

Using Graphql

```
query getContextData {
  contextData {
    id
    creationTime
    validFrom
    validUntil
    entityType
    regex
    context {
      key
      value
    }
  }
}
```

1.6.2. Setting context data

If you want to create a new context, you can omit the id if you want. If no id is set, the API will generate one for you using a UUID. If you use an id that is not yet in the system, this means you're creating a new context item.

From the UI

In the Context Data tab via the button *Add context data*. You need to be signed in for the button to be visible.

Using Graphql

```
mutation saveContextData {
  saveContextData(
    request: {id: "323b603d-5b5f-48d2-84fc-4e784e942289", entityType: TOPIC, regex:
".*collaboration", context: [{key: "app", value: "agoora"}, {key: "cost-unit", value:
"spoud"}, {key: "domain", value: "collaboration"}]}
  ) {
    id
    creationTime
    entityType
    regex
    context {
      key
      value
    }
  }
}
```

1.6.3. Removing context data

From the UI

Not available yet.

Using Graphql

```
mutation deleteContextData {
  deleteContextData(request: {id: "323b603d-5b5f-48d2-84fc-4e784e942289"}) {
    id
    creationTime
    entityType
    regex
    context {
      key
      value
    }
  }
```

```
}
```

1.7. Reprocess

Reprocessing applies the context rules **as they are now** to data you already have: for example after adding a rule for a service account or topic that showed up without context. Everything from the chosen start time on is rebuilt from the raw metrics; data before it is left as it is.

Prices don't need it: costs are computed when you look at them, from the pricing rules (with their dates) and the bills as they are now. A corrected price or a new bill shows at once.

While the rebuild runs, data from the start time on is incomplete and fills up again: depending on how much raw data there is, this takes minutes to hours. The `kcc_*` Prometheus gauges replay the historical values during that time.

What happens:

- Kafka Cost Control stops its Kafka Streams application.
- It decides where to start: the requested time, rounded down to the start of its aggregation window. If the raw topics no longer reach back that far (retention), it starts at the first complete window they still hold, so nothing is deleted that can't be rebuilt. Without a start time, it rebuilds everything the raw topics hold.
- It deletes the application's internal Kafka Streams topics (`<application id>-···-repartition` and `-changelog`). They hold the window state and the stream time; left in place, they made replayed windows count as already closed, and they were dropped.
- It deletes the stored windows (the OLAP table) from the start on, so the rebuilt windows replace them instead of being added next to them.
- It rewinds the raw topics to the start and the pricing rules to their beginning, wipes its local state and restarts (Kubernetes restarts the pod; you'll see the restart count go up).
- After the restart it processes the raw data again from the start with today's rules.

If deleting the internal topics fails (for example because the application's Kafka user isn't allowed to delete them), nothing else is changed and the application restarts with its data as it was; the reason is in the response and the logs.

Requirements and side effects:

- The application's Kafka user must be allowed to delete its own internal topics (prefix `<application id>-`) and to change its consumer group's offsets.
- The rebuilt windows are written to the `aggregated` and `aggregated-table-friendly` topics again. Consumers of those topics (e.g. a JDBC sink) receive new versions of the same keys and should upsert by key.
- Raw data older than the raw topics' retention can't be reprocessed.

1.7.1. Using the UI

- Go to the *Others* tab.
- Choose a start time (empty means everything the raw topics hold). The quick buttons above

help.

- Click *Reprocess* and confirm.

Using GraphQL

```
mutation reprocess {
  reprocess(areYouSure: "yes", startTime: "2024-01-01T00:00:00Z")
}
```

The answer says where the rebuild starts and how many internal topics and stored rows were deleted.

1.8. Deleting a metric

A metric you no longer need, e.g. one the collector stopped sending, keeps showing in pages, charts, exports and the assistant's answers for the time it was collected. Deleting it removes all its stored data. The raw topics aren't touched, so:

- a reprocess covering its time brings it back;
- if it is still being collected, new data keeps coming in. To stop it for good, remove it from the collector (e.g. Telegraf's `namepass`).

1.8.1. Using the UI

- Go to the *Others* tab. *Stored metrics* lists every metric with stored data and when it was last seen (*still arriving* when it is still collected).
- Click the bin next to the metric and confirm.

1.8.2. Using GraphQL

```
mutation deleteMetric {
  deleteMetric(metricName: "confluent_kafka_server_active_connection_count")
}
```

The answer is the number of stored rows deleted.

2. Installation

2.1. Prerequisites

This installation manual assumes that

1. You have a Kafka cluster
2. You have a schema registry
3. You have a Kubernetes cluster

2.2. Topics and AVRO schemas

Kafka cost control uses internal topic to compute pricing. You will have to create those topic before deploying the application. The documentation will show the default names, you can change them but don't forget to adapt the aggregator configuration.

2.2.1. Reference AVRO schemas

Some schemas will reference EntityType. Please add it to your schema registry and reference it when needed.

2.2.2. Topics

Topic name	Clean up policy	Key	Value
context-data	compact	*String*	ContextData
pricing-rules	compact	*String*	PricingRule
aggregated	delete	AggregatedDataKey	AggregatedDataWindowed
aggregated-table-friendly	delete	AggregatedDataKey	AggregatedDataTableFriendly
metrics-raw-telegraf-dev	delete	*None*	*String*

Context data

This topic will contain the additional information you wish to attach to the metrics. SEE TODO for more information. This topic is compacted and it is important that you take care of the key yourself. If you wish to delete a context-data you can set *null* as payload (and provide the key you want to delete).

Pricing rule

This topic will contain the price of each metric. Be aware that most of the metric will be in *bytes*. So if you want for example to have a price of *1.0$* per *GB* you will need to set the price to $1.0/1024^3$ = *0.000976276$* per *byte*. The key should be the metric name. If you wish to remove a price value, send the payload *null* with the key you want to delete. See TODO on how to use the API or the UI to set the price.

Aggregated

This topic will contain the enriched data. This is the result topic of the aggregator.

Aggregated table friendly

This is the exact same thing as *aggregated* except there are no hashmap and other nested field. Everything has be flattened. This topic makes it easy to sink the data into a table database.

Metrics raw telegraf

You can have multiple raw topics. For example one per environment or one per kafka cluster. The topic name is up to you, just don't forget to configure it properly when you deploy telegraf (see Kubernetes section). Give some special consideration to the `retention.ms` setting for the raw metrics topics. For example, if you want to distribute the cost of your monthly bill based on the raw metrics scraped over the course of the month then it is a good idea to retain the scraped data for more than 30 days. This gives people time to ask questions about their bill and also gives the opportunity to reprocess the metrics with new pricing rules/contexts if needed.

2.3. Kubernetes

You can find all the deployment files in the deployment folder. This folder use Kustomize to simplify the deployment of multiple instances with some variations.

The kubernetes deployment is in two parts. One part is the **kafka cost control software** (processing and UI) and the other part is the **kafka metric scrapper**. You may have multiple kafka metric scrapper deployment (one per kafka cluster for example), but you should need only one kafka cost control deployment.

2.3.1. Kafka metric scraper

This part will be responsible to scrape kafka for relevant metrics. Depending on what metrics you want to provide you will need a user with read access to kafka metric but also kafka admin client. Read permission is enough ! You don't need a user with write permission.

This documentation will assume that you use the `dev/` folder, but you can configure as much Kustomize folders as you want. The `dev/` folder is a good starting point if you have confluent cluster running: it runs Telegraf on Confluent's metrics API plus the kafka-scraper (partition and schema counts per topic).

To split a bill's partition line by tenant, build your overlay on the confluent-scraper folder: the same Telegraf plus the kafka-scraper, which counts each topic's partitions (`dev/` is built on it). Its scraper needs an API key that may describe topics, in the `kafka-cost-control-scraper-secret` secret; without a Schema Registry, set `CC_SCRAPE_SR_ENABLED=false` in a `kafka-cost-control-scraper-settings` config map.

If you only need Confluent's metrics, build your overlay on the confluent folder instead: Telegraf alone, with the credentials in the `telegraf-secret` secret. Its non-secret settings (`INPUT_URL`, `OUTPUT_BROKER`, `OUTPUT_TOPIC`, `ENVIRONMENT_NAME`) can go in a `telegraf-settings` config map, which takes precedence over the same keys in the secret, so they can live in git.

NOTE

Telegraf (1.38 and later) substitutes environment variables only inside quoted strings, so every setting is a plain value: no quotes, no brackets. `INPUT_URL` is a single URL; to read several clusters, repeat the parameter in it (`…?resource.kafka.id=lkc-a&resource.kafka.id=lkc-b`). To download only what costs are built from, add `metric` parameters to `INPUT_URL`, e.g. `&metric=io.confluent.kafka.server/request_bytes`; the sample `.env` lists the six metrics the Telegraf configs keep (network per principal and per topic, storage, partition count). Upgrading from an older version: rename `INPUT_URLS` to `INPUT_URL` and `OUTPUT_BROKERS` to `OUTPUT_BROKER`, and drop the brackets and quotes from those values and from `OUTPUT_TOPIC`.

Copy the environment sample file:

```
cd deployment/kafka-metric-scrapper/dev
cp .env.sample .env
vi .env
```

Edit the environment file with the correct output topic, endpoints and credentials.

Be sure to edit the **namespace** in the kustomization.yaml file.

Deploy the dev environment using kubectl

```
cd /deployment/kafka-metric-scrapper
kubectl apply -k dev
```

Wait for the deployment to finish and check the output topic for metrics. You should receive new data every minute.

2.3.2. Kafka cost control

For this part we will deploy the kafka stream application that is responsible to enrich the metrics (it stores them in its built-in DuckDB database, see Metric storage) and the UI to browse costs and define prices and contexts.

The base folder contains the manifests. Create your own Kustomize overlay next to it, for example `deployment/kafka-cost-control/my-env/kustomization.yaml`:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../base

namespace: kafka-cost-control

# the base leaves the Ingress hosts as TO_BE_DEFINED_BY_OVERRIDE, which Kubernetes
rejects
patches:
  - target:
      kind: Ingress
    patch: |-
      - op: replace
        path: /spec/rules/0/host
        value: kcc.example.com

generatorOptions:
  disableNameSuffixHash: true

secretGenerator:
- name: kafka-cost-control-secret
  envs:
  - .env
```

Put your credentials in a `.env` file in the same folder:

```
KAFKA_BOOTSTRAP_SERVERS=cluster.region.confluent.cloud:9092
CLUSTER_API_KEY=kafka-api-key
CLUSTER_API_SECRET=kafka-api-secret

KAFKA_SCHEMA_REGISTRY_URL=https://cluster.region.confluent.cloud
SCHEMA_REGISTRY_API_KEY=schema-registry-api-key
SCHEMA_REGISTRY_API_SECRET=schema-registry-api-secret

# Comma separated list of topics that contain raw metrics from telegraf
CC_TOPICS_RAW_DATA=metrics-raw-telegraf-dev

# Admin password
CC_ADMIN_PASSWORD=admin
```

Replace `kcc.example.com` with your own host. It is used by both the UI and the aggregator's `/graphql` endpoint.

Deploy the application using kubectl

```
cd deployment/kafka-cost-control
kubectl apply -k my-env
```

2.3.3. Helm Chart (Strimzi only)

If your Kafka cluster is managed by the Strimzi operators, you can use the provided Helm chart (under `helm/kcc-strimzi`) to deploy Cost Control with all the required components (Telegraf, Aggregator, UI) as well as all the required users and topics. The Helm chart still assumes that you already have a Schema Registry deployed. See the relevant README file for more information.

A component that is special to the Strimzi deployment is the Context Operator. This operator detects `KafkaUser` resources with a specific annotation and automatically creates a corresponding `Context` in Cost Control. This allows you to manage your contexts directly from Kubernetes. See the README in `strimzi-operator` for more information.

2.4. Metric storage

The aggregator can store the aggregated metrics in a built-in DuckDB database. Everything that shows a cost reads from it (Cost Overview, Explore, the unassigned topics and principals), as do the AI assistant and the exports below: costs are computed there when asked, from the pricing rules and the monthly bills. Without it, Kafka Cost Control shows usage but no costs. A stream of aggregated metrics is also always sent to the `aggregated` and `aggregated-table-friendly` topics, so you can still feed an external database of your choice from Kafka if you need to. These topics carry usage only: their `cost` field is empty.

2.4.1. DuckDB Integration

The provided Kubernetes manifests already enable it. If you deploy the aggregator yourself, set the following environment variables:

```
      containers:
        - name: kafka-cost-control
          image: spoud/kafka-cost-control:latest
          env:
            # enable the DuckDB integration
            - name: CC_OLAP_ENABLED
              value: "true"
            # path in the container where the DuckDB database will be stored
            # if this is not set, the data will be stored in-memory (i.e. it will be
lost when the container is restarted)
            - name: CC_OLAP_DATABASE_URL
              value: "jdbc:duckdb:/home/jboss/kafka-stream/duckdb.db"
```

Once the integration is enabled, you can export the collected metrics with their costs, one row per hour, topic or principal and metric, by using curl. With sign-in enabled (`cc.auth.mode basic`, the default, or `oidc`), the export needs it like the rest of the application: scripts use the `admin` user and its password (`cc.admin-password`), which also works when people sign in with OIDC. With `cc.auth.mode: none`, leave out `-u admin:<password>`:

```
# Get all metrics from the last 30 days in CSV format
curl -u admin:<password> -H "Accept: text/csv" http://localhost:8083/olap/export >
out.csv
# Get all metrics from the last 30 days in JSON format
curl -u admin:<password> -H "Accept: application/json"
http://localhost:8083/olap/export > out.json
# Get all metrics for all of February 2025 in CSV format
curl -u admin:<password> -H "Accept: text/csv"
"http://localhost:8083/olap/export?fromDate=2025-02-01T00:00:00Z&toDate=2025-03-
01T00:00:00Z" > out.csv
```

The usage values are stored; the costs (`cost`, `rate_cost`, `estimated`, `shared`, and the bills' `other` lines) are computed at the time of the request, from the pricing rules and bills as they are then. A month

exported before its bill was entered has estimated costs (`estimated` is true); export it again after entering the bill to get the billed ones.

Note that when using the `fromDate` and `toDate` parameters, the times must be specified in UTC in the ISO 8601 format (e.g. `2025-02-01T00:00:00Z`). Otherwise the API will return a 400 Bad Request error.

2.5. Troubleshooting

2.5.1. Kafka cost control is never ready

If the kafka-cost-control pod is never ready there are good chances that it is waiting on a topic before it can start. If you look closely in the log you will see a message like this:

```
2024-02-06 15:46:34,739 WARN  [io.qua.kaf.str.run.KafkaStreamsProducer] (pool-5-
thread-1) Waiting for topic(s) to be created: [non-existing-topic]
```

As soon as you create the missing topic(s), you should be good to go. Look again at the Topics section for more information on how to create a topic.

3. Architecture

3.1. Introduction and Goals

Many organizations have introduced Kafka either on premise or in the cloud in recent years.
Kafka platforms are often used as a shared service for multiple teams. Having all costs centralized in a single cost center means that there is no incentive to save costs for individual users or projects.

Kafka Cost Control gives organizations transparency into the costs caused by applications and allow to distribute platform costs in a fair way to its users by providing a solution that

- shows usage statistics per application and organizational unit
- allows defining rules for platform cost distribution over organizational units or applications
- works for most organizations, no matter if they use Confluent Cloud, Kubernetes or on-prem installations

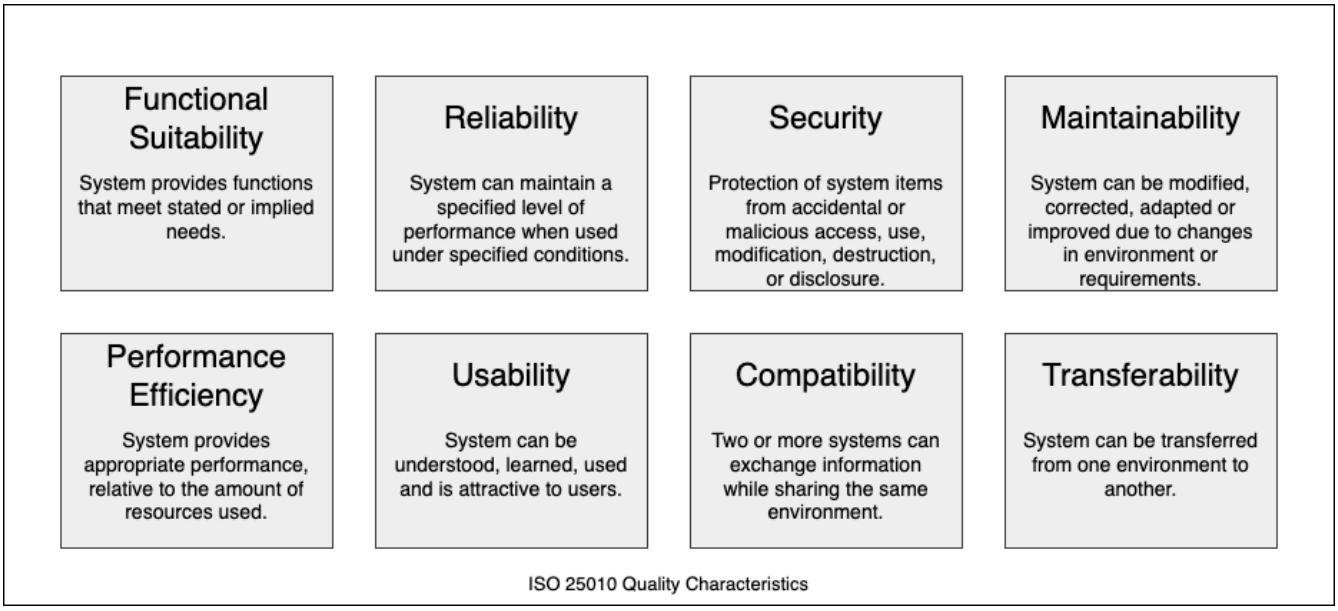
3.1.1. Requirements Overview

1. Collection and aggregation of usage metrics and statistics from one or multiple Kafka clusters. Aggregation by time:
    - hourly (for debugging or as a metric to understand costs in near real-time)
    - daily
    - weekly
    - monthly
2. Management of associations between client applications, projects and organizational units (OU)
    - automatic recognition of running consumer groups
    - automatic detection of principals/clients
    - creation, modification and deletion of contexts (projects and OUs)
    - interface to hook in custom logic for automatic assignment of clients to projects and OUs
    - manual assignment of auto-detected principals or consumer groups to projects and OUs
    - context can change in time, each item should have a start and end date (optional). This means that an item (ex a topic) can switch ownership at any point in time
3. Visualization of usage statistics
    - Costs and usage statistics can be broken down interactively
        - Summary view: total costs for timespan (day, week, month) per OU
        - Detail View OU by category: costs by category (produce, consume, storage) for the selected OU in the selected timespan
        - Detail View OU by application/principal/consumer-group/topic
    - Data must be made available in a format that can be used to display it with standard software (e.g. Kibana, Grafana, PowerBI), so that organizations can integrate it into an existing application landscape
    - provisioning of a lightweight default dashboard e.g. as a simple SPA, so that extra tooling is

not mandatory to view the cost breakdown
    - Items not yet classified should be easily identifiable, so we know what configuration is missing (for example a topic has no OU yet)
4. Management of rules, that describe how costs are calculated (aka pricing rules)
5. Management of rules, that describe how costs are calculated, e.g.
    - fixed rates for available metrics, i.e. CHF 0.15 per consumed GB
    - base charge, i.e. CHF 0.5 per principal per hour
    - rules can be changed at any time, but take effect at a specified start time
    - optional: backtesting of rules using historical data
6. Access Control
    - only authorized users can modify rules, OUs and projects
    - unauthenticated users should be able to see statistics
7. Observability
    - expose metrics so that the cost control app can be monitored
    - proper logging
8. Export of end-of-month reports as CSV or Excel for further manual processing
9. Ability to reprocess raw data in case a mistake was made. For example we see at the end of the month that an item was wrongly attributed to an OU. We should be able to correct this and reprocess the data.

3.1.2. Quality Goals

1. **Transferability / Extensibility**: Kafka Cost Control should be modular, so that company-specific extensions can be added.
   A core layer should contain common base functionality. Company specific terms or features should be separated into dedicated modules.
2. **Maintainability**: Reacting to changing requirements and implementing bug fixes should be possible within weeks.



ISO 25010 Quality Characteristics

3.1.3. Stakeholders

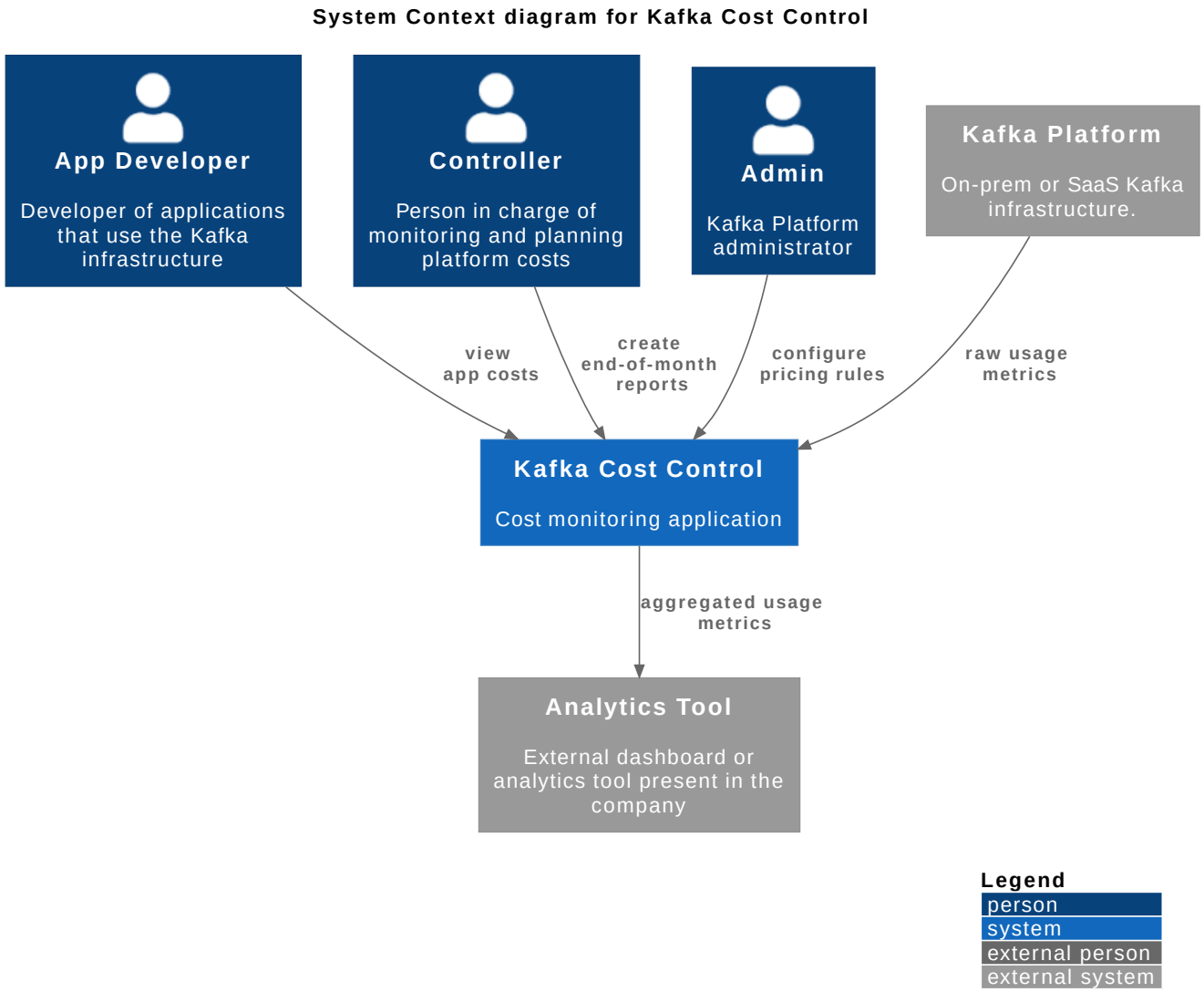
Role/Name	Expectations
Kafka user	Should be able to see their usage. Should take ownership of resources.
Management	Should have an overview of the costs and usage of Kafka.

3.2. Architecture Constraints

Constraint	Explanation
JVM based	use common language at SPOUD and many clients to make sure many can contribute
Hosting On-Site (not SaaS only)	Companies may not want to expose usage data to a SaaS provider

3.3. System Scope and Context

Kafka Cost Control is a standalone application that needs to integrate into an existing IT landscape.



3.4. Solution Strategy

3.4.1. Used Technologies

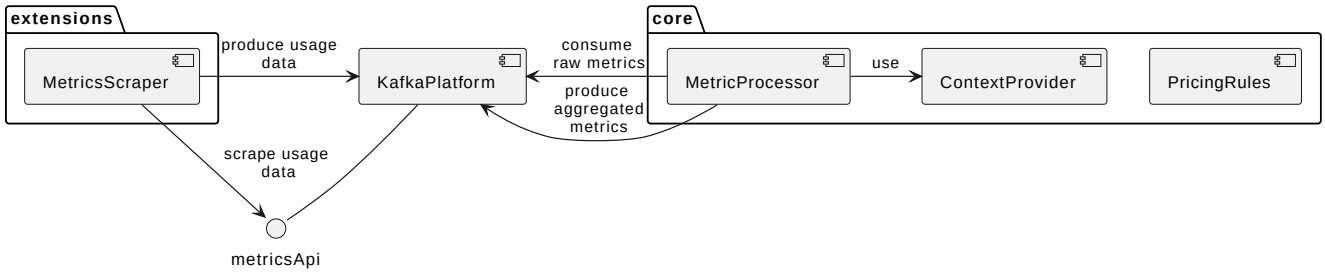
Technology	Reason
Telegraf	• Used for scraping metrics from data sources like Prometheus agents or Confluent Cloud API. • Versatile and lightweight tool that can be run in all environments. • supports Kafka
Kafka	for storing metrics, context info and pricing rules, reduces number of solution dependencies
Kafka Streams	for enriching metrics and storing pricing + context data into KTables
DataStore	A datastore, e.g. a SQL DB, will be used for the time based aggregations (e.g. end of month reporting). Avoids complex calendar logic in Kafka Streams.

3.4.2. Time based aggregations & scraping intervals

- **MetricsScraper** should ingest metrics with an interval of 1 minute for confluent cloud metrics. Other data sources can have longer intervals.
- **MetricsProcessor** aggregates metrics with short time windows of 60 minutes
  - variable cost is usually defined as *cost unit/minute*
  - The window value is the accumulated cost for one hour (interpolation may be needed when data points are missing)
  - this allows some tolerance for gaps in metrics and varying ingestion intervals

3.5. Building Block View

3.5.1. Whitebox Overall System



Building block	Description
PricingRules	Stores rules for turning usage information into costs
ContextProvider	Manages contextual information that can be used to enrich metrics with company-specific information. E.g. relations between clientIds, applications, projects, cost centers, …
MetricProcessor	• Defines interfaces for metrics, that must be used by *MetricsScraper* • Aggregates metrics into time buckets • Produces enriched data streams which includes contextual information
MetricsScraper	• uses a metric source, such as JMX or the confluent cloud metrics API to collect usage metrics • transforms the collected metrics into a format that is defined by *MetricProcessor*

3.5.2. MetricsScraper

Confluent Cloud

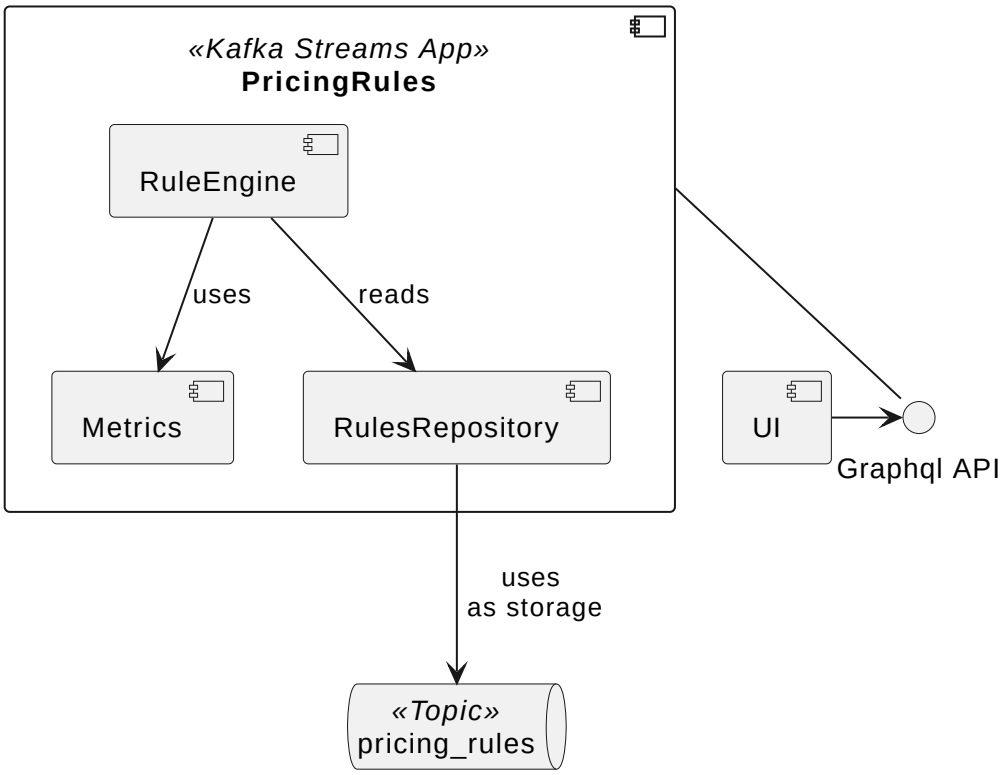
Confluent exposes many metrics in prometheus format. These will be scraped with telegraf. Some information are missing from the prometheus export endpoint and need to be fetched with custom queries/requests. This is done with a java application which exposes them as prometheus endpoint. Docs: https://docs.confluent.io/cloud/current/monitoring/metrics-api.html

Additional metric	endpoint/query
Partition count of a topic	`AdminClient` to list all topics, partition count and replication factor

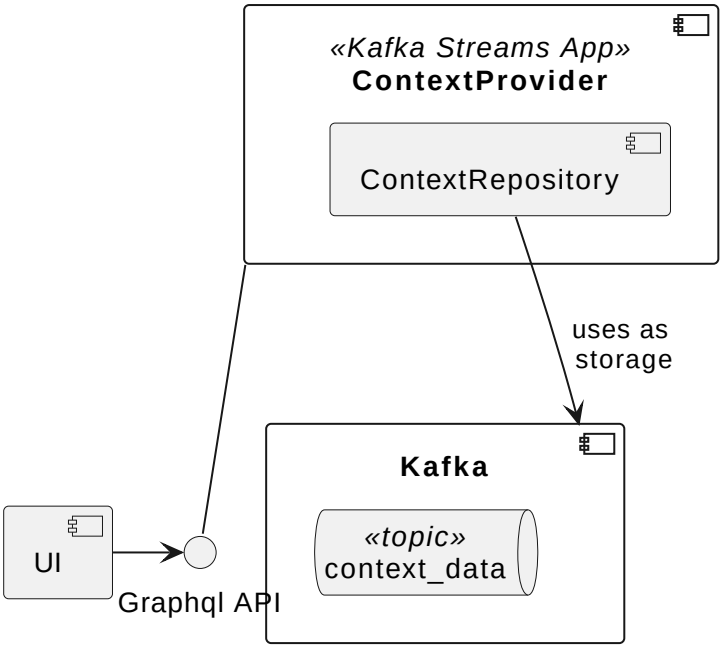
registered schemas for a topic	http requests to schema registry needed. 1. `GET {{schema_registry_url}}/schemas/` 2. Group by topic and create gauge metric for schema count by topic WARNING:: Does not account for soft deleted topics in confluent cloud! Possible sanity check: sum of all schemas should equal total number of schemas reported by `https://api.telemetry.confluent.cloud/v2/metrics/cloud/descriptors/metrics?resource_type=schema_registry` If the sum is smaller, then soft deletes were made (using the max version number might yield a better result in this case). WARNING:: If a non default naming strategy is used for subjects, then linking schemas/subjects to topics is not possible for all schemas.

[] | */github/workspace/images/.svg*

3.5.3. PricingRules



3.5.4. ContextProvider



Context format

- metrics are defined in the core
- a metric belongs to at least one of the dimensions
  - topic
  - consumer group
  - principal
- a context object can be attached to existing dimensions as a AVRO key-value pair to provide the needed flexibility

topic context as JSON record in a topic, record key="car-claims"

```
{
  "creationTime": "2024-01-01T00:00:00Z",
  "validFrom": "2024-01-01T00:00:00Z",
  "validUntil": null,
  "entityType": "TOPIC",
  "regex": "car-claims",
  "context": {
    "project": "claims-processing",
    "organization_unit": "non-life-insurance",
    "sap_psp_element": "1234.234.abc"
  }
}
```

topic context rule as JSON record in a topic, record key="default-rule-since-2020"

```
{
  "creationTime": "2024-01-01T00:00:00Z",
  "validFrom": "2024-01-01T00:00:00Z",
```

```
  "validUntil": null,
  "entityType": "TOPIC",
  "regex": "^([a-z0-9-]+)\\.([a-z0-9-]+)\\.([a-z0-9-]+)-.*$",
  "context": {
    "tenant": "$1",
    "app_id": "$2",
    "component_id": "$3"
  }
}
```

If naming conventions are very clear they could also be provided as a file / configuration.

principal context as JSON record in a topic, record key="cluster-id-principal-default-ctxt"

```
{
  "creationTime": "2024-01-01T00:00:00Z",
  "validFrom": "2024-01-01T00:00:00Z",
  "validUntil": null,
  "entityType": "PRINCIPAL",
  "regex": "u-4j9my2",
  "context": {
    "project": "claims-processing",
    "organization_unit": "non-life-insurance",
    "sap_psp_element": "1234.234.abc"
  }
}
```

INFO

Context objects will be started as AVRO messages. We use JSON as a representation here for simplicity.

Context Lookup

State stores in Kafka Streams will be used to construct lookup tables for the context.

The key is a string and is a free value that can be set by the user. If no key is provided the API should create random unique key. The topic is compacted, meaning if we want to delete an item we can send a null payload with its key.

Table 1. context lookup table

Key	Value
<type>_<cluster-id>_<principal_id>	<context-object>
PRINCIPAL_lx1dfsg_u-4j9my2_2024-01-01	{..., "regex": "u-4j9my2","context": {...}}
b0bd9c9a-08e6-46c7-9f71-9eafe370da6c	<context-object>

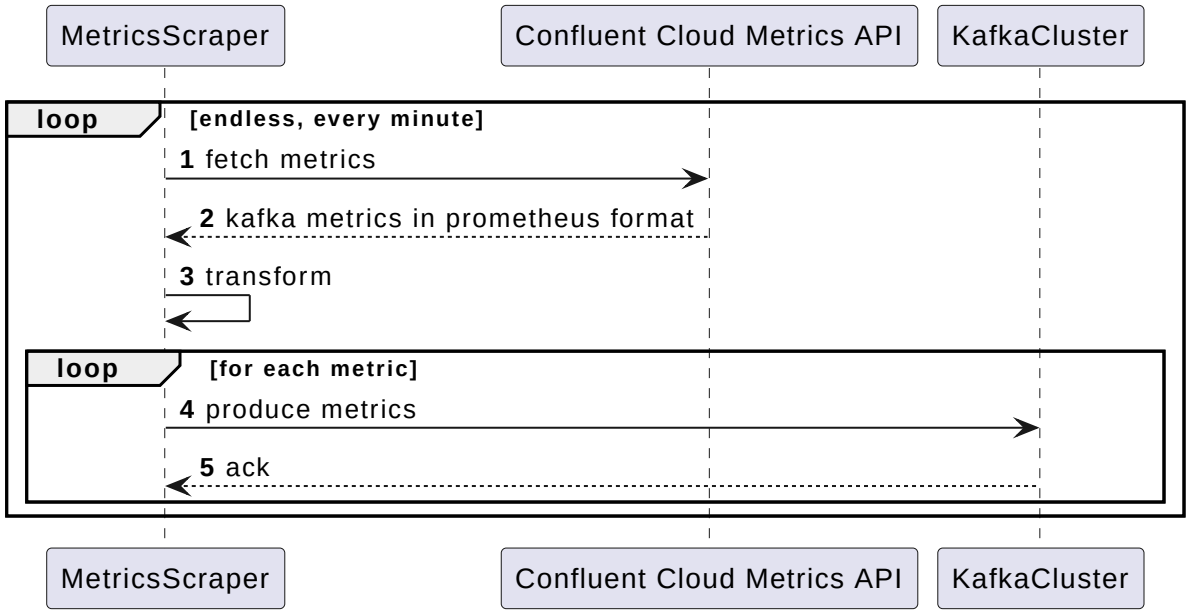
Once the table has been loaded, aggregated metrics can be enriched with a KTable - Streams join.

3.6. Runtime View

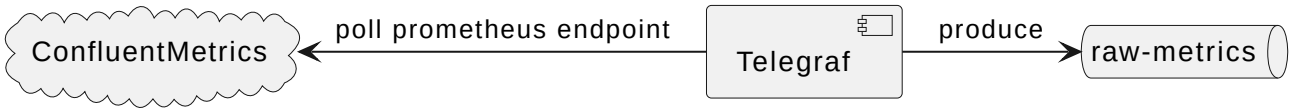
3.6.1. Metrics Ingestion from Confluent Cloud

Process to gather and aggregate metrics from Confluent Cloud.

The Confluent Metrics Scraper calls the endpoint `api.telemetry.confluent.cloud/v2/metrics/cloud/export?resource.kafka.id={CLUSTER-ID}` with Basic Auth in an interval of 1 Minute to obtain all metrics in Prometheus format.

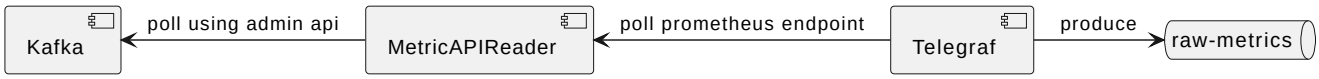


Telegraf is used to poll data using Confluent prometheus endpoint.



3.6.2. Metrics using Kafka Admin API

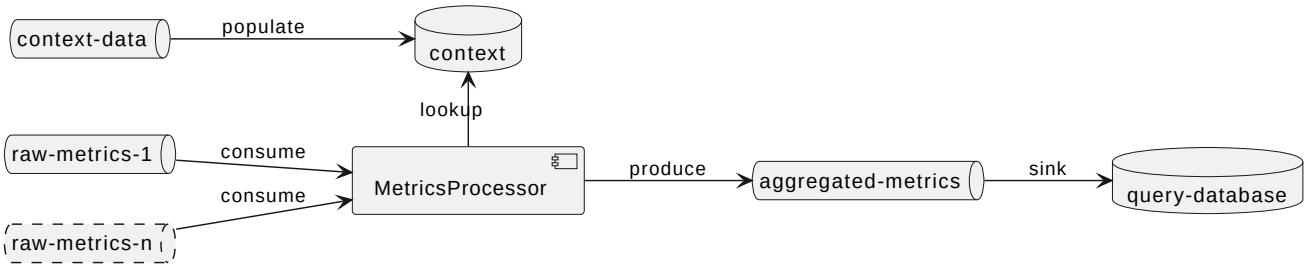
Some information can be gathered from the Kafka Admin API. We will develop a simple application that connect to the Kafka Admin API and expose metrics as prometheus endpoint. We can then reuse Telegraf to publish those metrics to kafka.



3.6.3. Other sources of metrics

Anyone can publish to the raw metrics topic. The metrics should follow the telegraf format. Recommendation: use one topic per source of metrics. The MetricEnricher application will anyway consume multiple raw metric topics.

3.6.4. Metrics Enrichment



1. Metrics are consumed from all the raw data topics.
2. Metrics are aggregated by the MetricsProcessor. Here we:
   - aggregate by hours
   - attach context
3. The aggregates are stored in the `aggregated-metrics` topic.
4. The aggregated metrics are stored into the query database.
5. Costs are not part of the aggregates: they are computed when queried, in the query database's `costs` view, from the pricing rules (with their dates) and the monthly bills. A price change or a new bill applies at once, without reprocessing.

The storage procedure into the query database must be idempotent in order to reprocess the enrichment in case of reprocessing.

Enrichment for topics

metric with topic name from confluent cloud

```
{
  "fields": {
    "gauge": 40920
  },
  "name": "confluent_kafka_server_sent_bytes",
  "tags": {
    "env": "sdm",
    "host": "confluent.cloud",
    "kafka_id": "lkc-abc123",
    "topic": "agoora-state-global",
    "url":
"https://api.telemetry.confluent.cloud/v2/metrics/cloud/export?resource.kafka.id=lkc-
abc123"
  },
  "timestamp": 1704805140
}
```

Enrichment for principals

metric with principal id from confluent cloud

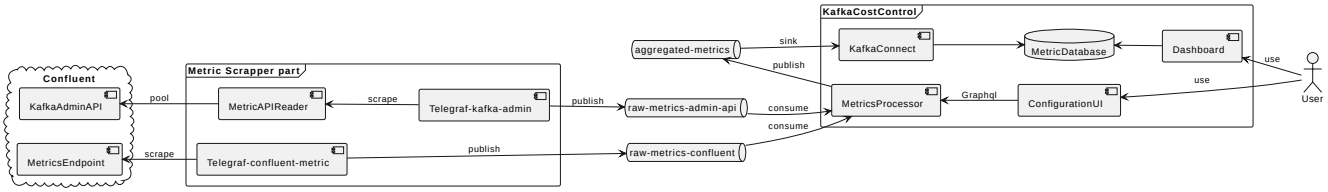
```
{
  "fields": {
    "gauge": 0
  },
  "name": "confluent_kafka_server_request_bytes",
  "tags": {
    "env": "sdm",
    "host": "confluent.cloud",
    "kafka_id": "lkc-abc123",
    "principal_id": "u-4j9my2",
    "type": "ApiVersions",
    "url":
"https://api.telemetry.confluent.cloud/v2/metrics/cloud/export?resource.kafka.id=lkc-
abc123"
  },
  "timestamp": 1704805200
}
```

3.6.5. Metrics Grouping

- confluent_kafka_server_request_bytes by kafka_id (Cluster) and principal_id (User) for the type Produce as sum stored in produced_bytes
- confluent_kafka_server_response_bytes by kafka_id (Cluster) and principal_id (User) for the type Fetch as sum stored in fetched_bytes
- confluent_kafka_server_retained_bytes by kafka_id (Cluster) and topic as min and max stored in retained_bytes_min and retained_bytes_max
- confluent_kafka_server_consumer_lag_offsets by kafka_id (Cluster) and topic as list of consumer_group_id stored in consumergroups

maybe more are possible.

3.7. Deployment View



3.8. Risks and Technical Debts

- Difficulty to get context data
  - Will the customer be willing to make the effort to provide the necessary data?
- Difficulty to put a set price on each kafka item
- How to integrate general cost like operation, etc. (not linked to a particular kafka item)
- Difficulty of integration with companies cost dashboard

3.9. Glossary

Term	Definition
OU	*Organization Unit*